Blog Posting Pipeline: Demo

A Deep Dive into a Fabricated Walkthrough of a Private System

Explainer for `blog-posting-pipeline-demo`

Read this box before anything else

This project is **not** an AI content pipeline. It is a static web page that *pretends* to be one, on purpose, and says so on every screen.
The real system, `blog-posting-pipeline`, is private. This repository is its public stand-in: a hand-written script of invented example runs for an invented company, replayed in a browser so that a reader can see the shape of the real architecture without the real system being exposed. Nothing this page "generates" was generated. No model is called. No network request is made. Every number it shows for a run was typed into a JavaScript file by hand.
One part of it *is* real, and that distinction is the whole point of the project: the "Blog Library" tab holds fifteen genuine, already-published articles from a real client site. Those were produced by the real pipeline. They are evidence. The scripted runs sitting next to them are not.

Contents

1 What this project is, and what it is not

1.1 The one-sentence version

The project in one sentence

A single-page, no-backend website that steps a reader through the seven stages of a private production content pipeline, using fabricated example data for an invented coffee brand, displayed alongside fifteen real published articles that the private pipeline actually produced.

1.2 Three things it is not

Because the page is designed to *look* like a working system, it is worth stating the negatives explicitly and early.

- **It is not the pipeline.** No part of the real system's code is in this repository. There is no Python, no prompt template, no client configuration, no scheduler, no API integration. The seven stages exist here only as text describing seven stages.
- **It is not a live demo.** Clicking "Run the pipeline" does not run anything. It reveals pre-written text on a timer. This is verified, not assumed: see Section 9, where the page was driven in a real browser and made zero network requests.
- **It is not client work on display.** The runs are for "Nordkast Coffee Supply," a brand that does not exist. The only real client work here is the fifteen library articles, which were already public before this repository existed.

Jargon: Static site

A website made only of files that a server hands over unchanged: HTML (the content), CSS (the appearance), JavaScript (the behaviour), and images. There is no program running on a server that builds a response when you visit. Everything that happens, happens inside your browser. That is why this demo can make no live calls: there is nothing on the other end to call.

Jargon: Pipeline

A program built as a chain of stages, where each stage's output is the next stage's input. The real system's chain is: decide a topic, pick keywords, plan sections, write the article, describe an image, make the image, publish it. Seven links in a chain.

2 Why a demo exists at all

2.1 The problem it solves

The owner has a production system that writes and publishes marketing articles for paying clients. He would like to show it to hiring panels. He cannot: per the project's own README, the pipeline *is* the agency's product, so its source is the business, not merely data that could be scrubbed clean and released.

That leaves an awkward gap. "Trust me, I built a seven-stage AI pipeline" is not evidence. A screenshot is barely better. So this repository answers a narrower question: can the *architecture* be shown honestly, in a form someone can click, without showing the implementation?

The trade being made

Give up all of the real system's substance (the prompts, the client rules, the code) and keep only its *shape* (seven stages, in order, with the right kind of data moving between them). Then be relentless about labelling the difference, because a convincing demo is exactly the thing most likely to be mistaken for the real product.

2.2 The honesty problem this creates

A fabricated demo that looks real is a liability unless the disclosure is impossible to miss. The repository takes this seriously and in several places overshoots into a defect, which Section 10 covers. The disclosure appears in at least six places:

- A permanent banner pinned across the top of the page, not dismissable.
- The page's introductory paragraph.
- The footer.
- A ribbon on the fabricated article card, separate from the banner.
- Captions under every photo and under the cost table.
- Comment headers at the top of every source file.

3 The provenance ladder

This is the most interesting idea in the project, and the thing worth defending in an interview. The page mixes content of three completely different origins in the same interface. Getting them confused would be the failure mode, so each tier is handled by a different mechanism and labelled differently.

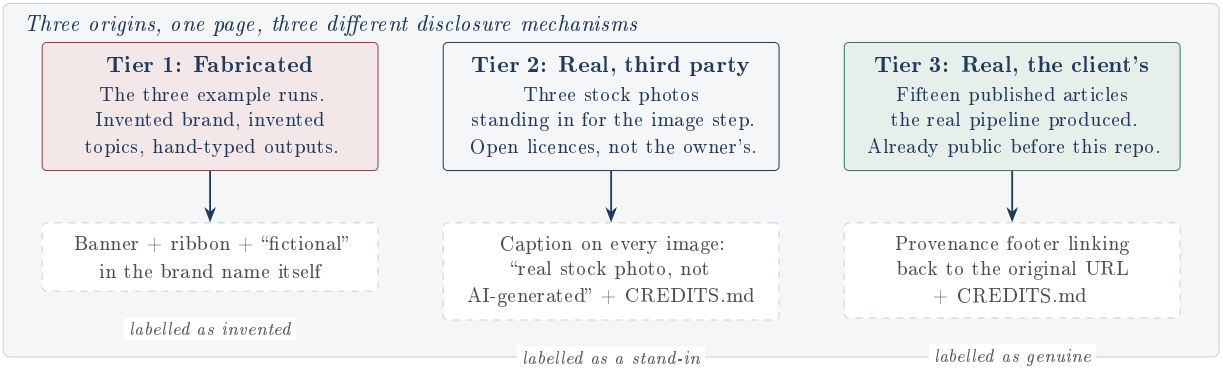

Figure 1: The provenance ladder. The design problem is not "is this labelled?" but "is each *kind* of content labelled in a way that survives being seen out of context?"

Why tier 3 is the clever part

The tempting design is to invent a fake blog library too, so that everything on the page is uniformly fictional and no disclosure question arises. The project rejects that. Its reasoning, from the README: the real client's posts are already public, the real pipeline genuinely produced them, so reproducing them (with a link back to the original) is *more* honest than inventing a fake library, because it is the only actual evidence on the page that the pipeline ships anything at all.
The cost of that choice is the risk this whole document is about: real and fake now sit side by side in one grid, and the labelling has to carry the entire burden.

4 Repository layout

Four hand-written source files, one of which is almost entirely data, plus assets.

```
blog-posting-pipeline-demo
├─ index.html   108 lines: page skeleton, banner, three panels
├─ styles.css   454 lines: design tokens copied from the live client site
├─ demo-data.js   687 lines / 231 KB: all the content, no logic
├─ app.js   458 lines: all the logic, no content
├─ netlify.toml   2 lines: "publish this folder as-is"
├─ LICENSE   PolyForm Strict (look, do not reuse)
├─ README.md
├─ favicon.svg, favicon.png
├─ assets/
│  ├─ fonts/
│  │  └─ space-grotesk.woff2   bundled, so no font CDN is called
│  ├─ images/   3 stock photos for the fabricated runs
│  │  └─ CREDITS.md   licence + author + change made, per photo
│  └─ posts/   15 real hero images from the client site
│     └─ CREDITS.md   why reproducing real client work is disclosed here
└─ docs/explainers/   this document
```

Figure 2: The whole repository. Note the split: `demo-data.js` is content with no behaviour, `app.js` is behaviour with no content.

Jargon: Asset

Any supporting file a page needs that is not code: an image, a font, a video. Kept in an `assets/` folder by near-universal convention.

Jargon: WOFF2 and WebP

Compressed file formats for the web. WOFF2 holds a font (here, the typeface "Space Grotesk"); WebP holds an image, at roughly half the size of an equivalent JPEG. Both are bundled locally rather than loaded from someone else's server, which is what lets the page claim it makes no outside requests.

5 Architecture

There is no server, no framework, and no build step. The browser loads three files and one of them starts drawing.

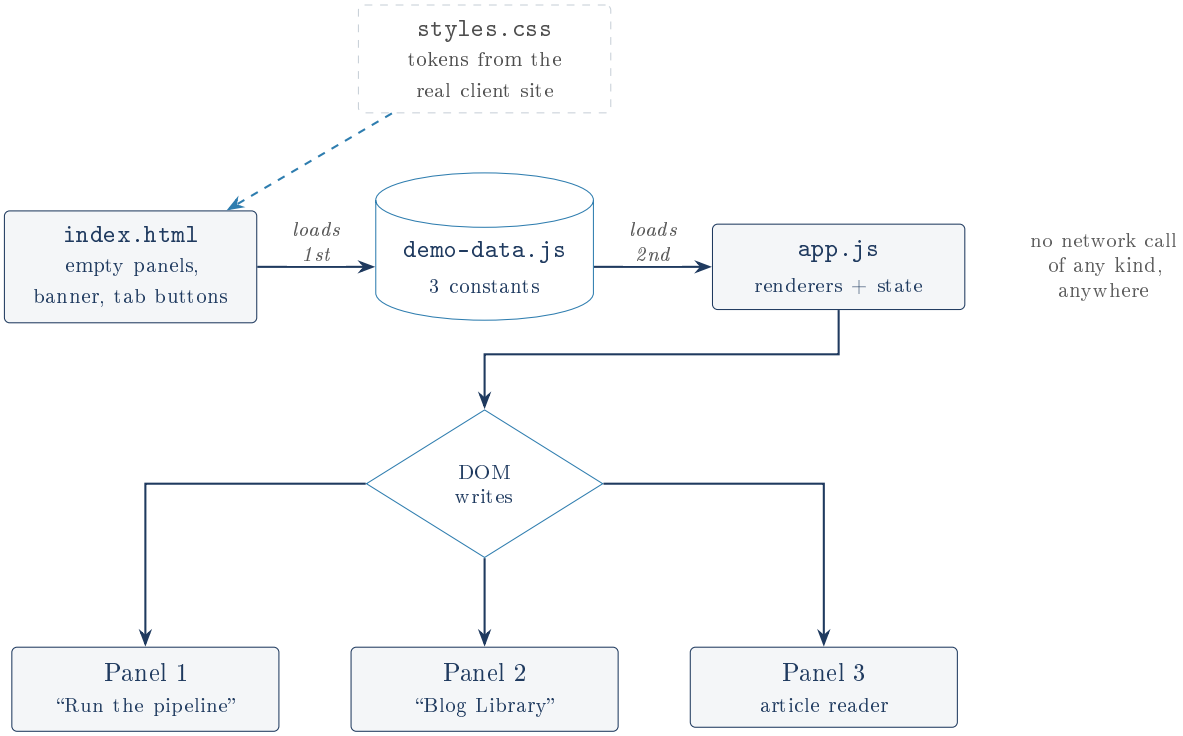

Figure 3: The entire architecture. `demo-data.js` must load before `app.js`, because `app.js` reads its constants at the moment the page finishes parsing.

Jargon: DOM (Document Object Model)

The browser's live, in-memory model of the page, shaped like a tree: the page contains a body, the body contains sections, a section contains a heading and paragraphs. JavaScript changes what you see by editing this tree. "Writing to the DOM" means changing the page in front of you.

Jargon: Vanilla JavaScript

JavaScript with no framework (no React, no Vue). The programmer manipulates the DOM directly. For a page this small it removes an entire build toolchain; the cost is that patterns a framework gives free, such as updating only the part of the page that changed, must be written by hand, and here one of them was not (Section 10).

5.1 The load-order dependency

At the bottom of `index.html`:

```
<script src="demo-data.js"></script>
<script src="app.js"></script>
```

Ordinary script tags, no `defer` or `async`, so the browser runs them in written order. `demo-data.js` declares three constants at the top level of the page's shared scope; `app.js` then reads them by name. Swap the two lines and the page breaks immediately, because `app.js` evaluates `PRESETS[0].id` while assigning its own first variable. This is a real, if conventional, coupling: the files communicate through global names rather than through any import mechanism.

6 The data layer: demo-data.js

231 KB across 687 lines, holding three constants and no functions. Every line of the provenance ladder from Section 3 shows up here as a separate structure.

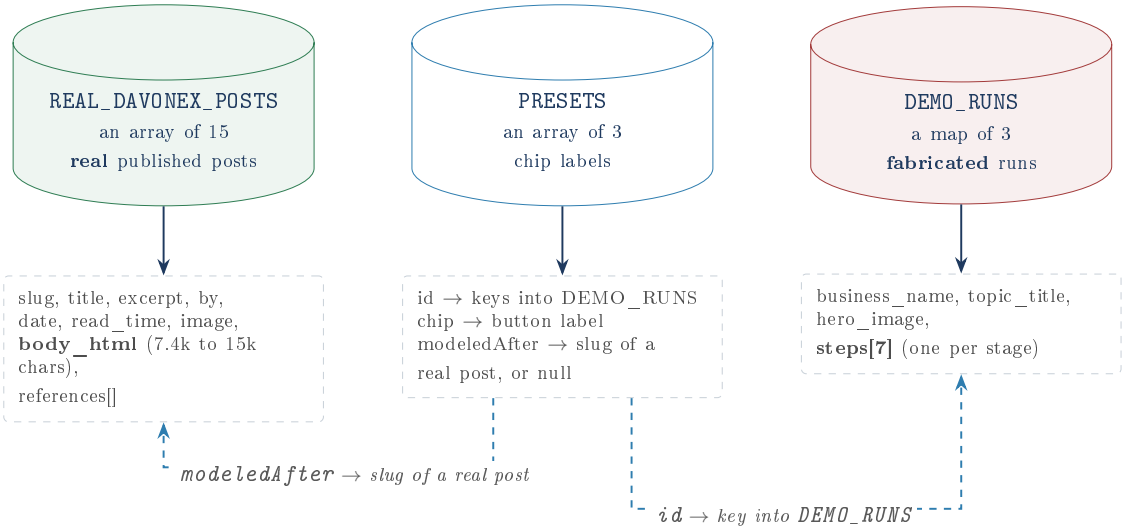

Figure 4: The three constants and how they reference each other. `PRESETS` is the join table: it links a fabricated run to the real post whose shape inspired it.

6.1 REAL_DAVONEX_POSTS: the evidence

Fifteen objects, each a faithful copy of a genuine published article: its title, excerpt, date, read time, a locally stored hero image, its full article body as HTML (between 7,358 and 15,078 characters; roughly 154,000 characters in total, which is most of this file's size), and its reference list.

This is a *snapshot*, taken 2026-07-07 and frozen. It is not fetched. The README is candid that this is the design's weak point: a real client publishes continuously, so the snapshot drifts out of date the moment it is taken, and the project accepted that rather than add the one live request that would break the "no network calls" guarantee.

Jargon: Snapshot

A copy of data taken at one instant and then kept, rather than re-read from the source each time. Fast and dependency-free; always at risk of being stale.

Jargon: Slug

The human-readable identifier in a web address. In `davonex.com/blog/what-is-answer-engine-optimization-for-e-commerce-2026/`, the long hyphenated part is the slug. Here it doubles as each post's primary key.

6.2 DEMO_RUNS: the fabrication

Three runs, keyed by id, each with exactly seven step objects. Each step carries an `id`, a display number such as `1 / 7`, a title, a model name, and a `type` that tells `app.js` which renderer to use. The `type` field is the dispatch key of the whole rendering layer:

type	Used by	Renders as
`json`	steps 1, 2, 3	a formatted, indented JSON block
`article`	step 4	the assembled article: intro, sections, FAQ, references
`text`	step 5	a single quoted paragraph (the image prompt)
`image`	step 6	a stock photo plus a usage block
`publish`	step 7	the full site preview plus a cost table

Jargon: JSON (JavaScript Object Notation)

A plain-text format for structured data, built from name/value pairs, lists, and nesting. It is how the real system's stages hand results to each other, and showing it raw is the demo's way of saying "a machine consumed this, not a human."

7 The seven stages

The rail of seven labels across the top is the demo's central claim: *this is the real system's architecture*. That claim is checkable, and it checks out.

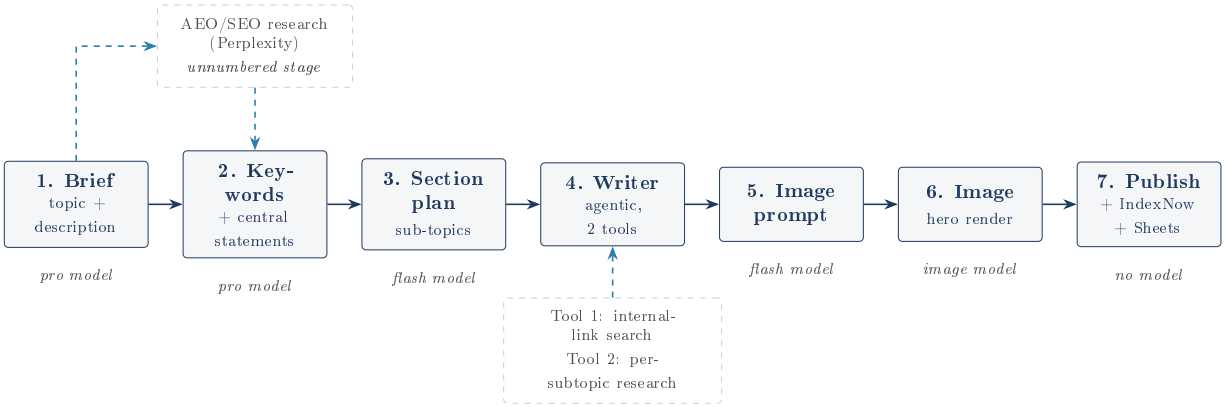

Figure 5: The seven stages as the demo presents them. The unnumbered research stage between 1 and 2 is not on the demo's visible rail, but it does appear as a line in the demo's cost table, which is a small and creditable piece of fidelity.

Architectural fidelity: verified

The demo's seven stages were compared against the real private project's own documentation. The stage names, their order, which stage uses the expensive "pro" model against the cheap "flash" model, the specific model identifiers, the fact that stage 4 is an agentic loop with exactly two tools, the unnumbered research stage between stages 1 and 2, and the publish stage's trio of actions (assemble, ping IndexNow, append a reporting row) all match.
This is the demo's strongest result. Its architectural claim is not marketing. It is accurate. The private details behind each stage are deliberately not reproduced here.

Jargon: Agentic loop

An arrangement where the language model is not asked one question and answered once. Instead it is given a set of *tools* it may call, and it loops: think, call a tool, read the result, think again, until it decides it is done. Stage 4's two tools search the client's own site for internal links, and research each sub-topic. This is why stage 4 is by far the most expensive

line in the cost table.

Jargon: SEO and AEO

SEO (Search Engine Optimization) is shaping a page so a search engine ranks it highly and a human clicks the link. AEO (Answer Engine Optimization) is shaping a page so an AI assistant that answers the question *directly* cites your page as its source. The distinction matters commercially: with AEO there may be no click at all, so the goal shifts from ranking to being extractable and quotable.

Jargon: IndexNow

A protocol for telling search engines "this URL just changed, come look now," rather than waiting for them to notice. One HTTP request. The demo shows this stage's status as `not sent (demo)`.

Jargon: Pinecone and embeddings

An *embedding* turns a piece of text into a long list of numbers positioned so that texts with similar meaning end up near each other. Pinecone is a database built to search those lists by nearness rather than by exact keyword. The real system uses it to remember what it has already written about, so it does not repeat an angle, and to find internal links. The demo only names it, in a cost row.

8 The logic layer: `app.js`

458 lines, no dependencies, organized as five labelled sections: step renderers, preset autofill, the pipeline run, the blog library, and tab switching, then an initializer.

8.1 Rendering by string concatenation

Every renderer builds an HTML string and assigns it to an element's `innerHTML`. There is no templating library and no virtual DOM. `renderStep` is the dispatcher:

```
function renderStep(run, step) {
  let body = "";
  if (step.type === "json") {
    if (step.input) body += `<p><strong>Input</strong></p>${renderJsonPanel(step.input)}...`;
    body += renderJsonPanel(step.output);
  } else if (step.type === "article") {
    body = renderArticle(step.article);
  } else if (step.type === "text") {
    body = `<p class="image-prompt">${escapeHtml(step.output)}</p>`;
  } else if (step.type === "image") {
    body = renderImageStep(run, step);
  } else if (step.type === "publish") {
    body = renderPublishStep(run, step);
  }
  ...
}
```

Jargon: innerHTML

A property of every element in the DOM. Read it and you get the element's contents as HTML text; *assign* to it and the browser discards everything inside that element and builds

the new contents from scratch. It is the quickest way to draw a chunk of page, and the reason for two of the defects in Section 10: it destroys, it does not update.

8.2 The escaping story, and where it stops

The file defines a small guard:

```
function escapeHtml(str) {
  return String(str)
    .replace(/&/g, "&amp;")
    .replace(/</g, "&lt;")
    .replace(/>/g, "&gt;");
}
```

Jargon: Escaping, and why it exists

If a chunk of text contains `<script>` and you paste it straight into a page, the browser will treat it as an instruction rather than as words. Escaping rewrites the dangerous characters into harmless stand-ins (< becomes `&lt;`) so they are *displayed* rather than *executed*. Skipping it is the root of the cross-site scripting (XSS) class of bugs.

It is applied to titles, prompts and FAQ text. It is deliberately *not* applied to authored HTML: `section.body` and `post.body_html` are injected raw, which is necessary, since those fields contain the tables and lists that are the whole point. The data is hand-written and static, so nothing hostile can reach it. Two observations remain, in Section 10: the function does not escape quote characters, yet its output is dropped into HTML attributes, and the same data is escaped in one renderer and not in another.

8.3 The run: a timer, not a state machine

`runPipeline` is the function behind the button. It is worth reading closely because it is the demo's only real piece of control flow, and it is much simpler than it appears on screen.

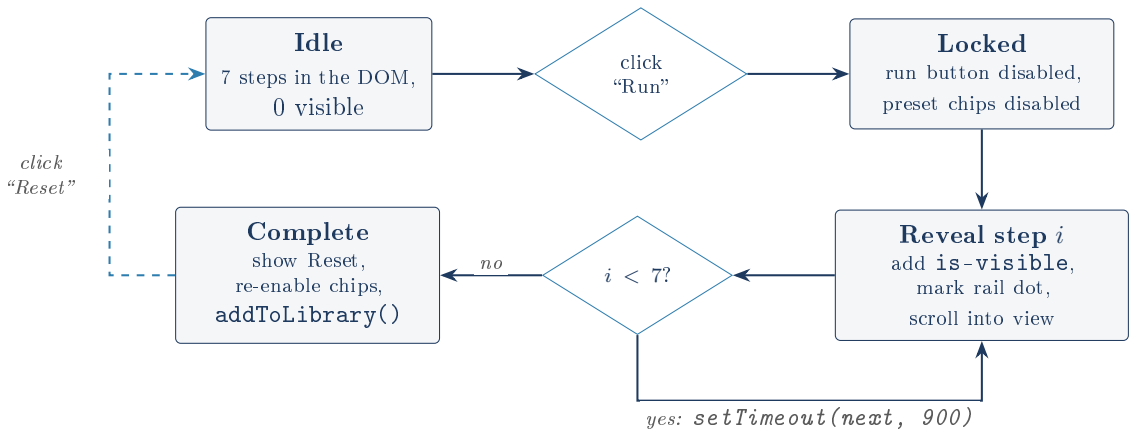

Figure 6: The run loop. All seven steps already exist in the DOM before the button is pressed; "running" only adds a CSS class to one more of them every 900 ms.

Two consequences follow from that design, and both are worth being able to state out loud:

- **The run cannot fail**, because nothing is computed. There is no error state in the diagram because there is no operation that could error. The real system's failure modes (a refusal, a timeout, a malformed JSON reply, a publish rejection) have no representation here at all.

- **The whole article is in the page from the start.** “Run the pipeline” reveals text the browser already holds. Anyone who opens the developer tools before clicking sees the finished post. This is not hidden and does not pretend otherwise, but it is the literal meaning of “scripted.”

Jargon: setTimeout

A browser function meaning “call this function once, after N milliseconds.” Chaining it to itself, as `next()` does, produces a sequence of delayed steps. Its sibling `setInterval` means “call this repeatedly, every N milliseconds, forever, until cancelled.”

8.4 The library card lifecycle

Finishing a run calls `addToLibrary`, which puts one fabricated card at the top of the grid of fifteen real ones, with a live countdown, for fifteen minutes.

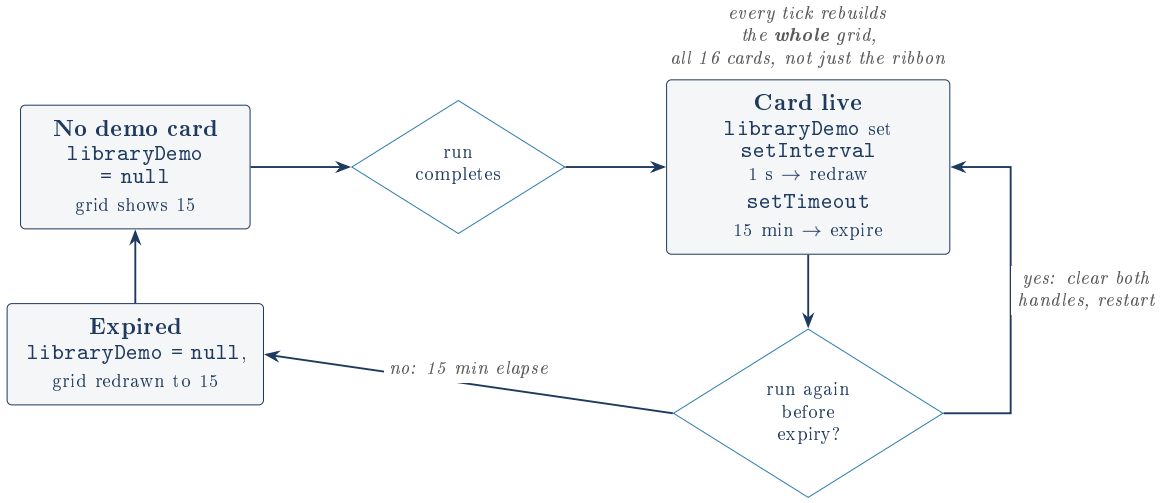

Figure 7: The demo card’s lifetime. The re-run path is handled correctly: both the interval and the timeout are cancelled before new ones are created, so timers cannot leak.

The cleanup is genuinely careful, and worth crediting:

```
function addToLibrary(runId) {
  if (libraryDemo) {
    clearInterval(libraryDemo.tickHandle);
    clearTimeout(libraryDemo.timeoutHandle);
  }
  ...
}
```

Without those two lines, running the pipeline three times would leave three intervals all redrawing the same grid every second, and three timeouts racing to delete a card. The author saw that. What the author did not see is what the interval does each time it fires, which is the subject of the next section.

9 What I verified by running it

The claims below were not taken on trust. The page was loaded in a real headless Chrome, driven through a full run, and instrumented.

Claim	Result	How it was checked
"No live API calls happen here"	**True**	Every network request the page made was recorded. Requests to non-local origins: **zero**.
No `fetch` or `XMLHttpRequest` in the source	**True**	Searched both JS files. No network API appears at all.
Fonts and images are local	**True**	The only `url()` in the CSS is the bundled WOFF2. No external stylesheet or CDN.
The page is free of JS errors	**True**	Zero page errors and zero console errors across load, a full run, and opening a post.
Seven stages step through	**True**	7 steps in the DOM, 0 visible before the click, 7 visible and 7 rail dots marked done after.
"All 15 real posts"	**True**	15 entries, 15 image files, every referenced path exists, no orphans.
A run adds exactly one demo card	**True**	Tab counter went from (15) to (16); one card carries the ribbon.
The countdown is live	**True**	Ribbon read 14:58, then 14:56 after 2.2 seconds.
Reader shows the real post + provenance	**True**	5,545 characters of prose, and a link back to the genuine original URL.
Cost range matches the real system	**Partly false**	See Section 10. Only one of the three presets matches.

The headline verification result

The demo's central promise, that it is inert and self-contained, is true, and provably so. Driven through its entire flow in a real browser, the page issued **no network requests whatsoever** and raised no errors. Every asset it needs is in the repository.

10 Honest findings

10.1 The cost claim is wrong for two of the three presets

This is the most serious finding in the project and the one most likely to be caught by an interviewer who reads carefully.

Under the cost table, on every run, the page prints:

> *"The total range is the real, published per-post cost for the production system (see its README). The per-step line items above are illustrative for this demo and are not the exact real breakdown."*

The second sentence is a good disclosure. The first sentence is a claim of fact, and it is only true for one of the three presets. The demo declares a different total range for each preset. Compared against the real private project's own published figure, the first preset's range matches it exactly; the other two presets' ranges do *not*. They extend past the real figure at both the low and the high end. They are fabricated variations, displayed under a caption asserting that they are the real published number.

The defect, stated plainly

The page labels its fabricated content scrupulously and its *real* content scrupulously, and then, in one place, labels fabricated content as real. Two of the three cost ranges are invented but captioned as "the real, published per-post cost."
The demo's own README compounds this by describing the headline figure as "anchored to the real, already-published production cost range from the source project's README." The range the README quotes is the spread across the demo's three presets, not the source project's published range. It reads as a citation and is not one.
This is fixable in minutes and should be fixed before anyone is shown the page: either set all three presets to the real published range, or change the caption to say the totals are illustrative too. The former is better, since a single honest number is worth more than three plausible ones. The owner should note that this is exactly the failure the rest of the project is built to prevent, which is worth saying first in an interview rather than being caught by it.

10.2 One preset's own arithmetic does not close

A smaller, related problem, found by adding up what is on screen. The third preset's per-step line items sum to $0.422, which is above the $0.42 upper bound of the total it prints directly underneath them. The other two presets' rows do sum to inside their stated ranges. A reader with a calculator finds a table that contradicts its own total by two tenths of a cent. The "illustrative" caption technically covers it, but it is careless, and it undermines the table's only job, which is to look like a real accounting.

10.3 The grid is destroyed and rebuilt every second, for fifteen minutes

`addToLibrary` puts a one-second `setInterval` on `renderLibrary`, so that the countdown ribbon ticks. But `renderLibrary` does not update the ribbon. It rebuilds everything:

```
function renderLibrary() {
  const grid = document.getElementById("library-grid");
  let html = "";
  if (libraryDemo) { html += demoPostCard(); ... }
  html += REAL_DAVONEX_POSTS.map(realPostCard).join("");
  grid.innerHTML = html;   // <- destroys and recreates all 16 cards, every tick
  ...
}
```

This was measured, not inferred. A mutation observer on the grid recorded **4 childList mutations in 4 seconds** while a demo card was live, and **0 in 4 seconds** when none was. Each rebuild recreates all 16 cards and all 16 `<img>` elements. Over a full fifteen-minute lifetime that is roughly 900 full teardowns of a grid that changed by five characters.

This is not merely inefficient, it is observable

While writing this document, an attempt to click a library card through the browser automation failed outright with "Node is detached from document." The card had been destroyed by the interval between being located and being clicked.

Real users mostly escape this by luck of implementation: the click handler is bound to the grid container rather than to the cards, so *clicks* survive the churn. What does not survive is anything the browser keeps on the element itself. A user cannot select or copy text from a card while the countdown runs, because the selection is wiped once a second. Hover transitions restart. On a low-end phone this is a needless 15-minute repaint loop.
The fix is small: give the ribbon its own element and have the interval write only its text, or, better, recompute the countdown only while the library panel is actually visible. As written, the timer keeps rebuilding a hidden grid the entire time the reader panel is open, which was also confirmed in the browser.

10.4 No tab looks selected while reading an article

The tab bar has two buttons, `pipeline` and `library`. The reader is a third panel, `panel-reader`, with no corresponding button. `switchTab("reader")` does this:

```
document.querySelectorAll(".tab").forEach(t =>
  t.classList.toggle("is-active", t.dataset.tab === tabId));
```

With `tabId` of `"reader"`, no button matches, so both buttons lose their active state. Confirmed in the browser: while reading a post, the number of tabs marked active is zero. The user is on a page whose navigation claims they are nowhere. Minor and cosmetic, but it is a state the tab component was never designed to represent, and the reader panel was bolted on afterwards without extending it.

10.5 `escapeHtml` does not escape quotes, and is used inside attributes

The function handles `&`, `<` and `>` but not `"` or `'`. Its output is then interpolated into attribute positions:

```
<img src="${post.image}" alt="${escapeHtml(post.title)}" loading="lazy">
```

A title containing a double quote would break out of the `alt` attribute. Today this cannot be exploited: every string comes from a hand-written constant in the repository, and no user input reaches any renderer. So it is not a live vulnerability. It is a guard that looks complete and is not, in a file where the next person to add a field will reasonably assume it is. Note also that `post.image` and `s.url` are interpolated into `src` and `href` with no escaping at all.

10.6 The same data renders two different ways

One renderer prints reference numbers explicitly, as `[1]` and `[2]`, taken from each source's `ref` field. The other, which shows the same sources a few hundred pixels lower inside the publish preview, drops `ref` entirely and lets an ordered list number them by itself. If the two ever disagreed, the step-4 view and the step-7 preview would cite the same article under different numbers. They agree today only because the `ref` values happen to run 1, 2 in order.

10.7 The articles claim citations they do not contain

The README describes the Writer step as producing "numbered citations with a References list," and lists "numbered citations" among the structural features deliberately matched to the real system's prompt templates. The reference lists are real and numbered. The inline citations are not there.

Searching all three fabricated articles' intro paragraphs and section bodies for a citation marker of the form `[1]` returns **zero matches** in all three. The prose cites nothing; a numbered list of sources simply appears at the end. This matters more than it looks: the whole editorial argument for AEO content is that claims are attributable, and the demo's articles assert statistics and

recommendations with no marker tying any of them to the two sources listed below. The `ref` field exists in the data and is rendered only in the list, never anchored in the text.

10.8 Smaller notes

- **Every fabricated source is a placeholder.** All six `sources` entries across the three runs point at `example.com`. That is the honest choice (inventing plausible real URLs would be worse) but it does mean the References lists, unlike the real posts' lists, lead nowhere.
- **Cards are not keyboard accessible.** A real post card is an `<article>` with a `data-open-post` attribute, opened by a delegated click. It has no `tabindex` and is not a button, so it cannot be reached or activated by keyboard. The demo card, which uses real `<a>` elements, can be. The seven rail dots are likewise plain `<div>`s.
- **An `aria-hidden` wrapper around a captioned image.** The demo card's image container is marked `aria-hidden="true"` while the `<img>` inside it carries a real `alt` description, which is then hidden from screen readers anyway.
- **One real post has an empty references array.** Handled correctly: the reader checks the length and omits the section rather than printing an empty heading.
- **`netlify.toml` is two lines** and says only "publish this folder." That is the correct amount of configuration for a site with no build step, and is worth noticing as a decision rather than an omission.
- **The page is marked `noindex`**, so search engines are told not to list it. Given that it reproduces a real client's articles verbatim, this is a deliberate and correct choice: it avoids competing with the client's own pages for the very search rankings the real pipeline exists to win.

11 What this demo cannot demonstrate

An interviewer's fair question is "what does clicking this actually prove?" The honest answer is: less than it appears, and the gap should be volunteered rather than defended.

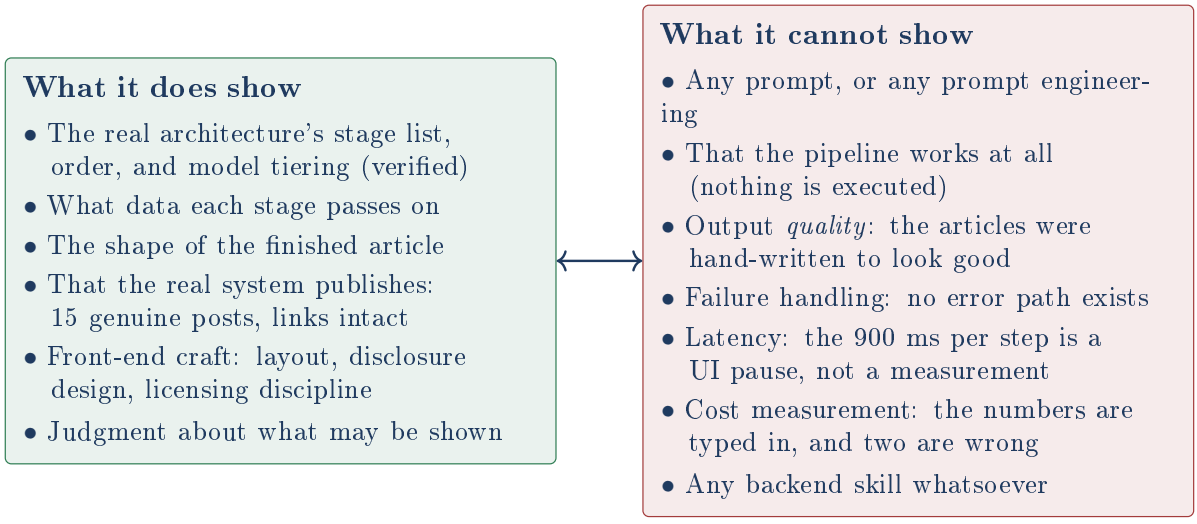

Figure 8: The boundary of the evidence. Everything on the right is a real part of the real system that this repository is structurally incapable of demonstrating.

The trap to avoid in an interview

The single worst thing that could be said about this page is: "here is my AI pipeline running." It is not running, and a sharp interviewer will find that out in one question, at which point every other claim becomes suspect.
The strong version of the same sentence is: "the real system is private, so I built an honest replica of its architecture and put it next to fifteen posts the real system actually published. The runs are fabricated and labelled as such. Here is what it proves and here is what it cannot." That framing turns the demo's greatest weakness, that it is fake, into evidence of the judgment that produced it.
The cost-caption defect in Section 10 should be fixed before that conversation happens, because it is the one place where the page's own discipline slips, and it slips in the exact direction the interviewer will be probing.

12 Licensing and content provenance

Three licences apply to one folder, which the repository handles correctly and unusually carefully.

Content	Licence	Note
The repository's own code and fabricated content	PolyForm Strict 1.0.0	Read it, do not reuse it. Appropriate for a portfolio piece.
`coffee-beans.webp`	Public domain	Credited anyway.
`espresso-machine.webp`	CC BY 2.0	Author credited, crop disclosed.
`coffee-cherries.webp`	CC BY-SA 4.0	**Share-alike**: `CREDITS.md` explicitly records that this one file stays CC BY-SA and does *not* fall under the repository's PolyForm licence.
The 15 real posts and images	The client's	Not open-licensed at all. Reproduced with a provenance link on every article and a `CREDITS.md` explaining why.

The share-alike catch, spotted

CC BY-SA is "viral": a work derived from it must carry the same licence. A cropped photo is a derivative. Dropping that photo into a strictly licensed repository without comment would have quietly violated its terms. `assets/images/CREDITS.md` carves it out by name. Most portfolio projects do not notice this. This one did, and can say so.

Jargon: PolyForm Strict

A licence that lets anyone read the source and use the software unmodified for noncommercial evaluation, while reserving essentially every other right. It is the standard choice for showing work publicly without granting a licence to reuse it.

13 The design layer

`styles.css` is 454 lines and its own honesty case. Its opening comment states that the palette, radii, shadows and the entire article template were copied from the real client's live public CSS and a real published page, on the reasoning that this is the visual design language every visitor's browser already renders, as distinct from the site's private server code, which is not in this repository.

The line the file draws is worth noting: the layout patterns are reused, but the demo's outer shell uses the invented "Nordkast Coffee Supply" mark, not the real client's logo or name, specifically to avoid impersonating the client's site. So the publish preview looks exactly like a real article on the real template, while being unmistakably branded to a company that does not exist.

The banner's styling carries an explicit comment that it is "deliberately not brand-blue" so that the disclosure never reads as decoration. That is a small choice that shows the disclosure was designed rather than added.

Jargon: Design tokens

Named values for the raw ingredients of a design: `--blue: #136BEE`, `--r-lg: 26px`. Defining them once and referring to them by name everywhere means the design stays consistent and can be re-themed by editing a dozen lines. The convention here is CSS custom properties declared on `:root`.

14 Summary

The verdict

As a piece of engineering this is a small, clean, dependency-free static page: about 1,700 lines of hand-written code, no build step, no framework, no network, no errors. That is appropriate. Nothing here needed to be harder.

As a piece of *judgment* it is more interesting than its size suggests. The provenance ladder, the licence carve-out, the refusal to hotlink, the `noindex`, and the decision to show real client work next to fabricated work rather than fake the whole library are all deliberate and defensible choices about what may honestly be shown.

Its architectural claim about the private system is accurate, and that was verified independently rather than taken on trust. Its cost claim is not accurate for two of three presets, which is a real defect in precisely the dimension the project stakes itself on, and it should be fixed before the page is shown to anyone.

Every finding in this document was taken from the source, or from driving the page in a real browser. Claims about the private `blog-posting-pipeline` were checked against that project's own documentation; none of its private detail is reproduced here.